

World Engine: Phase Development Plan

Item	Value
Version	1.0
Date	April 2026
Classification	Phase Development Plan
Extends	World Engine Architecture Specification v1.0
Extends	World Engine Pricing & Portfolio Addendum v1.0

Phase WE-1: Foundation — Types, Schema, and Intelligence Registry

Overview

Establish the World Engine's foundational data model: types, database tables, the intelligence registry, and the read-only API. Every subsequent phase writes to or reads from the structures built here. No operational intelligence is produced in this phase — it builds the skeleton.

Current State

The World Engine does not exist in the codebase. The following adjacent systems are relevant:

- **Organisational Graph Runtime:** `signal_architecture/graph/` — 6 node types, 6 edge types, 5 derivatives, PageRank propagation. Implemented in Python + Rust (`rust/dsi-core/`).
- **Assessment Database:** `infrastructure/db/models.py` — SQLAlchemy models for all assessment outputs. Every model cycle persists a versioned record with signal scores, tier assignments, and pricing outputs.

- **Analytics Module:** `infrastructure/analytics/` — Performance, portfolio, and signal analytics. Existing foundation for population-level computation.

Target State

A new top-level directory `world_engine/` containing the complete type system, database migrations, and intelligence registry with its read-only API.

File Structure

```
world_engine/
├── __init__.py
├── types.py                # Core data types (Relationship, LifecycleState,
├── registry/
│   ├── __init__.py
│   ├── store.py           # Intelligence registry data store
│   ├── api.py             # Read-only FastAPI router for registry queries
│   └── types.py           # Registry-specific request/response types
├── scanner/               # (Phase WE-3 – empty __init__.py placeholder)
├── inferencer/            # (Phase WE-3 – empty __init__.py placeholder)
├── validator/             # (Phase WE-3 – empty __init__.py placeholder)
├── lifecycle/             # (Phase WE-3 – empty __init__.py placeholder)
├── consistency/          # (Phase WE-2 – empty __init__.py placeholder)
├── portfolio/             # (Phase WE-5 – empty __init__.py placeholder)
├── causal_pricing/        # (Phase WE-4 – empty __init__.py placeholder)
└── scheduler.py           # (Phase WE-3 – placeholder)
```

Implementation Plan

WE-1a: Core Types

New file: `world_engine/types.py`

Define the complete type system using Pydantic models:

```
class LifecycleState(str, Enum):
    CANDIDATE = "candidate"
    PROVISIONAL = "provisional"
    ACTIVE = "active"
    DEPRECATED = "deprecated"

class CausalDirection(str, Enum):
    A_CAUSES_B = "a_causes_b"
```

```

B_CAUSES_A = "b_causes_a"
BIDIRECTIONAL = "bidirectional"
CONTEMPORANEOUS = "contemporaneous"

class DiscoveredRelationship(BaseModel):
    id: str # UUID
    source_signal: str # Signal feature ID (coverage-agnostic)
    target_signal: str # Signal feature ID
    direction: CausalDirection
    lag_months: float | None # Temporal lag (null for contemporaneous)
    correlation_rho: float # Spearman's rho at discovery
    granger_f_statistic: float | None # Granger causality F-stat
    granger_p_value: float | None
    effect_size: float # Cohen's d or equivalent
    confounders_tested: list[str] # Which confounders were controlled for
    holdout_rho: float | None # Replication correlation in holdout
    holdout_p_value: float | None
    predictive_hit_rate: float | None # Out-of-sample prediction accuracy
    population_size: int # N entities at discovery
    coverage_scope: list[str] # Which coverages contributed entities
    lifecycle_state: LifecycleState
    state_entered_at: datetime
    state_history: list[StateTransition]
    influence_weight: float # 0.0-1.0, computed from evidence streng
    created_at: datetime
    updated_at: datetime

class StateTransition(BaseModel):
    from_state: LifecycleState
    to_state: LifecycleState
    transitioned_at: datetime
    reason: str # Statistical evidence for transition
    evidence: dict # Raw metrics at transition time

class ConsistencyScore(BaseModel):
    entity_id: str
    assessment_id: str
    overall_consistency: float # 0.0-1.0
    signal_pair_scores: dict[str, float] # {pair_key: consistency}
    cross_layer_divergence: dict # {risk_vs_loss: float, risk_vs_exposure
    divergent_pairs: list[str] # Signal pairs with unexpected disagreeem
    computed_at: datetime

class CausalAdjustmentFactor(BaseModel):
    entity_id: str
    assessment_id: str
    caf_value: float # The multiplicative adjustment

```

```

confidence: float # 0.0-1.0
active_precursors: list[PrecursorEvaluation]
trajectory: TierMigrationDistribution
relationships_evaluated: int
constrained: bool # Whether floor/cap was applied
raw_caf: float # Pre-constraint value
computed_at: datetime

class PrecursorEvaluation(BaseModel):
    relationship_id: str
    precursor_signal: str
    entity_value: float # Entity's current value
    threshold: float # Relationship's trigger threshold
    distance_from_threshold: float # How far into precursor state
    implied_probability: float # P(consequence) given this precursor
    lag_months: float

class TierMigrationDistribution(BaseModel):
    current_tier: int
    probabilities: dict[int, float] # {tier: probability}
    expected_tier: float # Weighted average
    policy_period_months: int

class PortfolioConcentration(BaseModel):
    entity_id: str # Commercial entity (portfolio owner)
    dimension: str # node | pathway | derivative | cohort
    detail: str # Specific concentration identified
    severity: float # 0.0-1.0
    affected_entities: list[str] # Entity IDs in the concentration cluste
    computed_at: datetime

```

WE-1b: Database Schema

New migration: alembic/versions/010_world_engine_tables.py

Tables:

Table	Purpose	Key Columns
we_relationships	Discovered causal relationships	All fields from DiscoveredRelationship
we_state_transitions	Lifecycle audit trail	relationship_id FK, from/to state, evidence JSON
we_consistency_scores	Per-assessment	entity_id, assessment_id, scores

	consistency	JSON
<code>we_causal_adjustments</code>	Per-assessment CAF	<code>entity_id</code> , <code>assessment_id</code> , <code>caf_value</code> , trajectory JSON
<code>we_portfolio_concentrations</code>	Portfolio concentration alerts	<code>entity_id</code> (commercial), <code>dimension</code> , <code>severity</code>
<code>we_scan_runs</code>	Batch run audit trail	<code>run_id</code> , <code>started_at</code> , <code>completed_at</code> , stats JSON

Indexes: `we_relationships` indexed on `lifecycle_state` + `source_signal` + `target_signal`. `we_consistency_scores` indexed on `entity_id` + `computed_at`. `we_causal_adjustments` indexed on `assessment_id`.

WE-1c: Intelligence Registry Store

New file: `world_engine/registry/store.py`

The registry is the single interface through which other DSI components access World Engine intelligence. It wraps the database with a query-optimised read API and a write API used only by internal Engine subsystems.

```
class IntelligenceRegistry:
    """Single point of access for all World Engine intelligence."""

    # Read API (available to all DSI components)
    def get_active_relationships(self, signal_ids: list[str] | None = None) -> list[Relationship]
    def get_relationships_for_entity(self, entity_id: str, signal_ids: list[str] | None = None) -> list[Relationship]
    def get_consistency_score(self, assessment_id: str) -> ConsistencyScore | None
    def get_caf(self, assessment_id: str) -> CausalAdjustmentFactor | None
    def get_portfolio_concentrations(self, commercial_entity_id: str) -> list[PortfolioConcentration]

    # Write API (internal - used only by Engine subsystems)
    def register_candidate(self, relationship: DiscoveredRelationship) -> str
    def transition_state(self, relationship_id: str, to_state: LifecycleState, event: str) -> bool
    def store_consistency_score(self, score: ConsistencyScore) -> None
    def store_caf(self, caf: CausalAdjustmentFactor) -> None
    def store_concentration(self, concentration: PortfolioConcentration) -> None
```

WE-1d: Registry API Routes

New file: `world_engine/registry/api.py`

FastAPI router mounted at `/api/v1/world-engine/` :

Endpoint	Method	Description
<code>/relationships</code>	GET	List relationships, filterable by state, signal, coverage
<code>/relationships/{id}</code>	GET	Single relationship with full history
<code>/consistency/{assessment_id}</code>	GET	Consistency score for an assessment
<code>/caf/{assessment_id}</code>	GET	CAF for an assessment
<code>/portfolio/{entity_id}/concentrations</code>	GET	Portfolio concentrations for a commercial entity
<code>/stats</code>	GET	Engine statistics: relationship counts by state, scan run history

WE-1e: Integration Points

Mount the registry router in `infrastructure/api/main.py` :

```
from world_engine.registry.api import router as world_engine_router
app.include_router(world_engine_router, prefix="/api/v1/world-engine", tags=["Wor
```

Add Engine health to existing health check endpoints.

Dependencies

- Alembic migration infrastructure (exists)
- FastAPI app (exists)
- SQLAlchemy models (exists — extend pattern)

Success Criteria

1. All database tables created via migration
2. Registry store passes unit tests for all read/write operations
3. API routes return correct responses (empty — no data yet)

4. `world_engine/` importable from all other DSI modules
 5. No changes to existing pricing or commercial code
-

Phase WE-2: Consistency Scorer

Overview

The first operational World Engine subsystem. Computes cross-signal and cross-layer consistency for every assessment. This produces immediate value (anti-gaming, confidence enrichment) and generates the structured divergence data that the discovery engine (Phase WE-3) requires.

Current State

The three-layer assessment runs in parallel (risk, exposure, loss) on the same signals with different weights. But no system currently measures whether the three layers agree, or whether signals within and across categories tell a consistent story.

Target State

Every assessment produces a `ConsistencyScore` stored in the registry. The consistency score is visible in the frontend workbench alongside existing confidence metrics. The Consistency Scorer runs inline (during the assessment, not as a batch process) to ensure every assessment has a consistency score.

Implementation Plan

WE-2a: Consistency Computation Engine

New file: `world_engine/consistency/scorer.py`

```
class ConsistencyScorer:
    """Computes intra-assessment consistency metrics."""

    def score(self, assessment: AssessmentResult) -> ConsistencyScore:
        """
        Compute consistency for a completed assessment.

        Three computations:
```

1. Signal-pair consistency: Do signals within the same group agree?
 2. Cross-group consistency: Do signal groups tell a consistent story?
 3. Cross-layer consistency: Do risk, exposure, and loss layers agree?
- """

Signal-pair consistency: For each pair of signals within a signal group, compute the normalised distance between their scores. Signals expected to correlate positively (based on being in the same group) should show convergence. Divergence beyond a threshold is flagged. The output is a per-pair consistency score (0–1) and a list of divergent pairs.

Cross-group consistency: For each pair of signal groups, compute the normalised distance between their group-level composite scores. Groups that are structurally related (e.g., Technical Infrastructure and Corporate Digital Footprint) should show moderate agreement. Groups that are structurally independent (e.g., Network Authority and Public Records) provide the most valuable consistency signal — agreement between independent sources is strong evidence of accuracy.

Cross-layer consistency: Compare the risk tier, exposure band, and loss propensity band. An entity that is Tier 1 (excellent) on risk but High on loss propensity has a significant cross-layer gap. This is not necessarily wrong — it could reflect genuine divergence between current state and forward trajectory. But it is informative and must be surfaced.

Overall consistency score: Weighted average of the three components: signal-pair (0.3), cross-group (0.4), cross-layer (0.3). The cross-group weight is highest because it measures agreement between genuinely independent data sources.

WE-2b: Inline Integration

Files to modify:

- `layers/risk/workflow.py` — Add consistency scoring as a post-assessment step

After the three-layer assessment completes (Steps 5a/5b/5c) and before pricing (Step 8), insert a consistency scoring step. This step is non-blocking — if the consistency scorer fails, the assessment proceeds with no consistency score attached.

```
# In workflow.py, after three-layer assessment
try:
    consistency_scorer = ConsistencyScorer()
    consistency_score = consistency_scorer.score(assessment_result)
    registry.store_consistency_score(consistency_score)
    assessment_result.consistency_score = consistency_score.overall_consistency
    assessment_result.divergent_pairs = consistency_score.divergent_pairs
except Exception:
    assessment_result.consistency_score = None # Graceful degradation
```

WE-2c: Assessment Output Extension

Files to modify:

- `layers/risk/types.py` — Add `consistency_score: float | None` and `divergent_pairs: list[str] | None` to assessment result types
- `infrastructure/api/types.py` — Add consistency fields to API response schemas

WE-2d: Frontend — Consistency Display

New file: `frontend/src/components/submissions/Workbench/ConsistencyCard.tsx`

A card component that displays:

1. **Overall consistency score** as a gauge (0–1), with colour coding: green (>0.8), amber (0.5–0.8), red (<0.5)
2. **Divergent pairs** listed if any exist, with the specific signals and their scores
3. **Cross-layer summary:** A mini-table showing risk tier vs exposure band vs loss band with agreement/divergence indicators

Files to modify:

- `frontend/src/components/submissions/Workbench/SummaryTab.tsx` — Add `ConsistencyCard` to summary view
- `frontend/src/components/submissions/Workbench/RiskTab.tsx` — Add consistency section to risk assessment tab
- `frontend/src/stores/dsiStore.ts` — Add consistency score to store

WE-2e: Seed Script

Files to modify: `seed_dsi_bench.py`

Update seed script to run consistency scoring for all seeded assessments, populating the `we_consistency_scores` table.

Constraints

1. Consistency scoring must add < 50ms to assessment time (it operates on already-computed scores, no new extraction)
2. Failure must not block the assessment pipeline

3. No changes to scoring logic, tier assignment, or pricing

Success Criteria

1. Every seeded assessment has a consistency score in the database
 2. ConsistencyCard renders in the frontend with real data
 3. Divergent pairs are correctly identified for assessments with known disagreements
 4. Assessment pipeline latency increase < 50ms
 5. All existing tests pass unchanged
-

Phase WE-3: Discovery & Validation Pipeline

Overview

The autonomous offline process that discovers emergent cross-signal causal relationships. This is the core intelligence engine that justifies the World Engine's existence. It operates as a batch process, completely separate from the real-time pricing pipeline.

Current State

No discovery or validation capability exists. The intelligence registry (Phase WE-1) and consistency scorer (Phase WE-2) provide the foundation.

Target State

A fully autonomous batch pipeline that runs periodically, mines the assessment database for cross-signal relationships, validates them statistically, and manages their lifecycle without human intervention.

Implementation Plan

WE-3a: Correlation Scanner

New file: `world_engine/scanner/correlation_scanner.py`

```
class CorrelationScanner:
```

```

"""Mines the assessment database for cross-signal correlations."""

def scan(self, min_entities: int = 50) -> list[CandidateRelationship]:
    """
    1. Load all signal scores from assessment database
    2. Build entity × signal matrix (entities as rows, signals as columns)
    3. Include signals from ALL coverages (coverage-agnostic)
    4. For each signal pair, compute Spearman's rho
    5. Filter: |rho| > 0.3, p < 0.01
    6. Exclude known designed relationships (same signal group = expected)
    7. Return candidates
    """

```

Key design decisions:

- The scanner is **coverage-agnostic**. An entity assessed for both Cyber and D&O contributes signals from both coverages. A cross-coverage correlation (e.g., a cyber technical signal correlating with a D&O governance signal) is exactly the kind of relationship that makes the Engine valuable.
- The scanner uses Spearman's rho (rank correlation), not Pearson's. This handles non-linear monotonic relationships without assuming normality.
- Signal pairs within the same signal group are excluded — they are already designed to correlate by virtue of being in the same group with similar weights.
- Results are stored as `CandidateRelationship` objects with full statistical evidence.

WE-3b: Causal Inferencer

New file: `world_engine/inferencer/granger.py`

```

class CausalInferencer:
    """Determines causal direction for correlated signal pairs."""

    def infer(self, candidates: list[CandidateRelationship]) -> list[DirectedCand
        """
        For each candidate:
        1. Load time-series data for both signals (requires ≥ 2 assessments per e
        2. Apply Granger causality test in both directions
        3. Assign direction: A→B, B→A, bidirectional, or contemporaneous
        4. Estimate lag (in months) from the optimal lag order
        """

```

New file: `world_engine/inferencer/confound_control.py`

```

class ConfoundController:
    """Eliminates spurious relationships explained by confounders."""

    CONFOUNDERS = ["entity_size_band", "industry_code", "geography", "assessment_

def filter(self, candidates: list[DirectedCandidate]) -> list[DirectedCandida
    """
    For each candidate:
    1. Compute partial correlation controlling for each confounder
    2. If  $|\text{partial\_rho}| < 0.2$  after controlling, discard (spurious)
    3. Record which confounders were tested
    """

```

WE-3c: Validation Engine

New file: world_engine/validator/holdout.py

```

class HoldoutValidator:
    """Tests relationship replication in unseen data."""

    def validate(self, candidate: DirectedCandidate, holdout_fraction: float = 0.
        """
        1. Split entity population 70/30 (training/holdout)
        2. Verify relationship replicates in holdout:  $|\text{rho}| > 0.2$ ,  $p < 0.05$ 
        3. Record holdout statistics
        """

```

New file: world_engine/validator/stability.py

```

class TemporalStabilityTracker:
    """Tracks relationship stability over rolling windows."""

    def check(self, relationship_id: str, min_windows: int = 3) -> StabilityResul
        """
        1. Compute correlation in each observation window
        2. Require persistence across min_windows consecutive windows
        3. Flag if correlation sign flips or magnitude drops below threshold
        """

```

New file: world_engine/validator/predictive.py

```

class PredictiveValidator:
    """Tests out-of-sample prediction accuracy."""

```

```

def validate(self, relationship: DiscoveredRelationship) -> PredictiveResult:
    """
    1. Identify entities where precursor signal recently crossed threshold
    2. Track whether target signal follows within the relationship's lag wind
    3. Compute hit rate (predictions that materialised / total predictions)
    """

```

WE-3d: Lifecycle Manager

New file: world_engine/lifecycle/manager.py

```

class LifecycleManager:
    """Autonomous state machine for relationship lifecycle."""

    def evaluate_all(self) -> list[StateTransition]:
        """
        For each relationship in the registry:
        - CANDIDATE: Check holdout + stability → promote to PROVISIONAL or discard
        - PROVISIONAL: Check predictive validation → promote to ACTIVE or demote
        - ACTIVE: Check ongoing predictive power → maintain or deprecate
        - DEPRECATED: Check if re-discovered in latest scan → potentially re-promote

        All transitions are autonomous. No human gate.
        """

        PROMOTION_CRITERIA = {
            "candidate_to_provisional": {
                "holdout_rho_min": 0.2,
                "holdout_p_max": 0.05,
                "stability_windows_min": 3,
            },
            "provisional_to_active": {
                "predictive_hit_rate_min": 0.6,
                "effect_size_min": 0.3,
                "min_age_months": 6,
            },
            "active_to_deprecated": {
                "predictive_hit_rate_below": 0.4,
                "consecutive_windows_below": 3,
            },
        }

```

WE-3e: Scheduler

New file: world_engine/scheduler.py

Orchestrates the batch pipeline:

```
class WorldEngineScheduler:
    """Orchestrates the discovery-validation-lifecycle pipeline."""

    def run_cycle(self) -> ScanRunReport:
        """
        Full cycle:
        1. CorrelationScanner.scan() → candidates
        2. CausalInferencer.infer() → directed candidates
        3. ConfoundController.filter() → cleaned candidates
        4. HoldoutValidator.validate() → validated candidates
        5. Registry.register_candidate() for surviving candidates
        6. LifecycleManager.evaluate_all() → state transitions
        7. Store ScanRunReport for audit
        """

    def should_run(self) -> bool:
        """
        Run if:
        - New assessments since last run exceed threshold (default: 100)
        - OR time since last run exceeds max interval (default: 7 days)
        - AND minimum population size met (default: 50 entities with ≥ 2 assessments)
        """
```

WE-3f: Unit Tests

New directory: tests/unit/test_world_engine/

Test File	Tests
test_correlation_scanner.py	Synthetic data with known correlations; verify detection and filtering
test_granger.py	Synthetic time series with known causality; verify direction assignment
test_confound_control.py	Synthetic data with known confounders; verify elimination
test_holdout.py	Split validation with known replicable/non-replicable relationships
test_lifecycle.py	State machine transitions with mocked validation results
test_scheduler.py	Full pipeline with synthetic data

Constraints

1. **No impact on real-time pipeline:** The entire phase runs as offline batch
2. **Statistical library:** Use `scipy.stats` for Spearman/Granger, `statsmodels` for partial correlation. No custom statistics.
3. **Minimum population:** Scanner refuses to run with < 50 entities with temporal data. No discovery on sparse datasets.
4. **Audit everything:** Every scan run, every candidate, every state transition persisted with full evidence

Dependencies

- Phase WE-1 complete (types, registry, database)
- Phase WE-2 complete (consistency data enriches discovery)
- Sufficient assessment population in database (seed script or real data)

Success Criteria

1. Scanner discovers known correlations in synthetic seeded data
 2. Lifecycle manager correctly promotes/demotes based on evidence thresholds
 3. Full pipeline runs end-to-end on seed data without errors
 4. All transitions recorded in `we_state_transitions` with evidence
 5. No changes to existing pricing, scoring, or assessment code
-

Phase WE-4: Causal Graph Pricing (CAF)

Overview

The parallel pricing track. For every assessment, the World Engine evaluates the entity against the validated causal graph and produces a Causal Adjustment Factor (CAF) that enters the premium formula as a multiplicative adjustment.

Current State

The pricing formula is: $P = (B \times R) \times ILF \times D \times M_{risk} \times M_{loss} \times M_{exp}$

All modifiers are computed inside the pricing engine from YAML-configured bands. There is no mechanism for external intelligence to influence the premium.

Target State

The formula becomes: $P = (B \times R) \times ILF \times D \times M_{risk} \times M_{loss} \times M_{exp} \times CAF$

The CAF is computed by the World Engine, arrives at the pricing engine as an external input, and appears as a distinct line item in the premium audit trail. When the Engine has no active relationships or insufficient confidence, CAF defaults to 1.0 (zero impact).

Implementation Plan

WE-4a: Causal Pricing Engine

New file: world_engine/causal_pricing/engine.py

```
class CausalPricingEngine:
    """Computes the Causal Adjustment Factor for an entity assessment."""

    CAF_FLOOR = 0.80
    CAF_CAP = 1.50
    CONFIDENCE_GATE = 0.6
    MIN_RELATIONSHIPS = 2

    def compute_caf(
        self,
        entity_id: str,
        signal_scores: dict[str, float],
        derivative_values: dict[str, float],
        current_tier: int,
        tier_rates: dict[int, float],
        policy_period_months: int = 12,
    ) -> CausalAdjustmentFactor:
        """
        1. Query registry for active relationships matching entity's signals
        2. Evaluate precursor state for each relationship
        3. Synthesise tier migration trajectory
        4. Compute CAF from trajectory
        5. Apply constraints (floor, cap, confidence gate)
```

```
"""
```

New file: world_engine/causal_pricing/trajectory.py

```
class TrajectoryEngine:
    """Synthesises forward risk trajectory from precursor evaluations."""

    def compute_trajectory(
        self,
        precursors: list[PrecursorEvaluation],
        current_tier: int,
        policy_period_months: int,
    ) -> TierMigrationDistribution:
        """
        Combine multiple precursor probabilities into a single trajectory.

        Method: Each precursor implies a probability of the target signal degrading.
        Map target signal degradation to composite score impact.
        Map composite score impact to tier migration probability.
        Combine multiple independent precursors using probability union.
        """
```

WE-4b: Pricing Engine Integration

Files to modify:

- layers/risk/workflow.py — Add CAF computation after three-layer assessment, before pricing
- layers/risk/pricer.py — Accept CAF as input, multiply into final premium
- layers/risk/types.py — Add CAF fields to AssessmentResult and PricingResult

In workflow.py (after three-layer assessment, before pricing):

```
# Compute CAF (non-blocking)
try:
    causal_engine = CausalPricingEngine()
    caf_result = causal_engine.compute_caf(
        entity_id=entity_id,
        signal_scores=signal_scores,
        derivative_values=derivative_values,
        current_tier=risk_tier,
        tier_rates=tier_rate_map,
    )
    registry.store_caf(caf_result)
```

```
except Exception:
    caf_result = CausalAdjustmentFactor.neutral() # CAF = 1.0
```

In pricer.py — the final premium calculation gains one additional multiplier:

```
# Existing
final_premium = base_premium * ilf * ded_factor * m_risk * m_loss * m_exp

# New
final_premium = base_premium * ilf * ded_factor * m_risk * m_loss * m_exp * caf_r

# Audit trail
pricing_result.caf = caf_result.caf_value
pricing_result.caf_confidence = caf_result.confidence
pricing_result.caf_detail = caf_result # Full provenance
```

WE-4c: API Response Extension

Files to modify:

- `infrastructure/api/types.py` — Add CAF fields to pricing response
- `infrastructure/api/routes/` — Ensure CAF data flows through existing assessment endpoints

The assessment API response gains:

```
{
  "pricing": {
    "final_premium": 23895,
    "static_premium": 20250,
    "caf": {
      "value": 1.18,
      "confidence": 0.74,
      "relationships_evaluated": 3,
      "active_precursors": 2,
      "constrained": false,
      "trajectory": {
        "current_tier": 2,
        "probabilities": {"1": 0.02, "2": 0.65, "3": 0.25, "4": 0.08},
        "expected_tier": 2.39
      }
    }
  }
}
```

WE-4d: Frontend — CAF Display

New file: `frontend/src/components/submissions/Workbench/CausalAdjustmentCard.tsx`

Displays within the Pricing Anatomy tab:

1. **CAF value** with colour coding: green (<1.0 = credit), neutral (1.0), amber (1.0–1.2), red (>1.2)
2. **Trajectory visualisation:** Horizontal bar chart showing tier migration probabilities
3. **Active precursors:** Expandable list showing which causal relationships fired, with entity's values vs thresholds
4. **Static vs Causal premium comparison:** Side-by-side showing P_{static} and P_{final}

Files to modify:

- `frontend/src/components/submissions/Workbench/PricingTab.tsx` — Add `CausalAdjustmentCard`
- Premium Assembly tab — Show CAF as a line item in the waterfall

WE-4e: Seed Script

Files to modify: `seed_dsi_bench.py`

Seed synthetic active relationships in the registry, then run CAF computation for all seeded assessments. Some entities should have active precursors ($\text{CAF} \neq 1.0$) and some should not ($\text{CAF} = 1.0$), demonstrating both states.

Constraints

1. **CAF defaults to 1.0** when: no active relationships exist, confidence < 0.6 , fewer than 2 relationships evaluated. This ensures the pricing pipeline is unaffected at the Seed maturity stage.
2. **CAF is constrained** to $[0.80, 1.50]$. These bounds may widen autonomously as predictive accuracy is demonstrated.
3. **No modification to existing modifiers:** M_{risk} , M_{loss} , M_{exp} remain exactly as configured. CAF is additive.
4. **Full audit trail:** Both P_{static} and P_{final} are recorded. The CAF provenance chain is independently auditable.

Dependencies

- Phase WE-1 complete (types, registry)
- Phase WE-3 at least partially complete (active relationships in registry)
- Existing pricing engine (workflow.py, pricer.py)

Success Criteria

1. Seeded assessments with active precursors show CAF $\neq$ 1.0
 2. Seeded assessments without precursors show CAF = 1.0
 3. Premium audit trail shows both P_static and P_final with full CAF provenance
 4. Frontend displays CAF card with trajectory visualisation
 5. All existing pricing tests pass (CAF defaults to 1.0 when no relationships exist)
 6. Calibration harness still passes for all coverages
-

Phase WE-5: Portfolio Intelligence & Simulation

Overview

Assemble individual Organisational Graphs into a Portfolio Graph. Detect hidden concentration risk. Enable scenario simulation. This is the capability that transforms DSI from a risk-level assessment tool into a portfolio-level intelligence platform.

Current State

The Organisational Graph Runtime (`signal_architecture/graph/`) builds per-entity graphs. No system assembles these into a portfolio view or analyses cross-entity relationships.

Portfolio analytics (`infrastructure/analytics/portfolio.py`) exists but operates on assessment scores, not on graph structures.

Target State

A Portfolio Graph that connects all assessed entities within a commercial entity's portfolio. Concentration intelligence published to the registry. A Scenario Simulation API that

propagates hypothetical shocks through the causal graph and estimates portfolio impact.

Implementation Plan

WE-5a: Portfolio Graph Builder

New file: world_engine/portfolio/graph_builder.py

```
class PortfolioGraphBuilder:
    """Assembles individual Organisational Graphs into a Portfolio Graph."""

    def build(self, commercial_entity_id: str) -> PortfolioGraph:
        """
        1. Load all assessed entities for this commercial entity
        2. Load each entity's Organisational Graph
        3. Infer inter-entity edges:
            a. Shared technology nodes (same cloud provider, same CDN, etc.)
            b. Shared partner nodes (same supplier, same certifier)
            c. Shared jurisdiction nodes (same regulatory environment)
            d. Causal pathway correlation (entities sharing active precursor patte
        4. Compute portfolio-level derivatives (aggregate entropy, velocity, drif
        5. Run portfolio-level PageRank to identify systemic nodes
        """
```

New file: world_engine/portfolio/types.py

```
class PortfolioGraph(BaseModel):
    commercial_entity_id: str
    entity_count: int
    nodes: list[PortfolioNode]          # Union of all entity nodes + inter-enti
    edges: list[PortfolioEdge]          # Intra-entity + inter-entity edges
    systemic_nodes: list[SystemicNode]   # Nodes with high portfolio-level PageRa
    aggregate_derivatives: dict[str, float] # Portfolio-level entropy, velocity,
    built_at: datetime

class SystemicNode(BaseModel):
    node_id: str
    node_type: str
    label: str
    portfolio_pagerank: float           # Importance in the portfolio graph
    connected_entities: list[str]       # Which portfolio entities depend on thi
    failure_impact_estimate: float      # Estimated portfolio impact if this nod
```

WE-5b: Concentration Detector

New file: world_engine/portfolio/concentration.py

```
class ConcentrationDetector:
    """Identifies hidden concentration risk in the Portfolio Graph."""

    def detect(self, portfolio: PortfolioGraph) -> list[PortfolioConcentration]:
        """
        Four dimensions:
        1. Node concentration: Single external nodes connected to > threshold ent
        2. Pathway concentration: Multiple entities sharing active causal precurs
        3. Derivative correlation: Portfolio-wide derivative trends (systemic dri
        4. Cohort concentration: Over-representation of specific signal-derived c
        """
```

WE-5c: Appetite Integration

Files to modify:

- infrastructure/models/commercial_schema.py — Add registry query to
evaluate_appetite()

When appetite is evaluated for a new submission, query the registry for portfolio concentrations. If accepting this risk would increase concentration on a flagged dimension, attach a concentration warning to the appetite result. This does not auto-decline — it flags.

```
def evaluate_appetite(coverage, submission_data, entity):
    # ... existing appetite checks ...

    # Portfolio concentration check (new)
    concentrations = registry.get_portfolio_concentrations(entity.id)
    for conc in concentrations:
        if submission_would_increase(conc, submission_data):
            result.warnings.append(ConcentrationWarning(
                dimension=conc.dimension,
                detail=conc.detail,
                current_severity=conc.severity,
            ))
    return result
```

WE-5d: Scenario Simulation API

New file: world_engine/portfolio/simulation.py

```
class ScenarioSimulator:
    """Propagates hypothetical shocks through the Portfolio Graph."""
```

```

def simulate(self, scenario: ScenarioDefinition, portfolio: PortfolioGraph) -
    """
    1. Apply shock to specified nodes/signals in the portfolio graph
    2. Propagate through causal edges (from active relationships)
    3. Compute per-entity impact (implied signal degradation, tier migration)
    4. Aggregate to portfolio level (loss distribution, concentration amplifi
    5. Identify mitigation paths (which entities to remediate/non-renew)
    """

class ScenarioDefinition(BaseModel):
    name: str
    description: str
    shocks: list[Shock]

class Shock(BaseModel):
    target_type: str                # "signal", "node", "derivative", "exter
    target_id: str                  # Signal ID, node ID, or event descripti
    magnitude: float                # Score change, or 0.0 for complete fail
    propagation: str               # "direct_only", "one_hop", "full_cascad
    decay_rate: float              # Propagation strength decay per hop (de

class SimulationResult(BaseModel):
    scenario: ScenarioDefinition
    entity_impacts: list[EntityImpact] # Per-entity: affected signals, implied
    aggregate_loss_estimate: LossRange # Expected portfolio loss range
    concentration_amplification: float # How much worse due to concentration (1
    mitigation_paths: list[MitigationPath]
    computed_at: datetime

```

New API routes (added to world engine router):

Endpoint	Method	Description
/portfolio/{entity_id}/graph	GET	Portfolio Graph summary
/portfolio/{entity_id}/graph/systemic-nodes	GET	Highest-PageRank nodes in portfolio
/portfolio/{entity_id}/simulate	POST	Run scenario simulation
/portfolio/{entity_id}/simulate/presets	GET	Pre-defined scenario templates

WE-5e: Frontend — Portfolio Dashboard

New files:

- `frontend/src/components/portfolio/PortfolioDashboard.tsx` — Top-level portfolio view
- `frontend/src/components/portfolio/ConcentrationPanel.tsx` — Concentration alerts with dimension breakdown
- `frontend/src/components/portfolio/SystemicNodesPanel.tsx` — Portfolio PageRank leaders
- `frontend/src/components/portfolio/ScenarioPanel.tsx` — Scenario simulation interface

Portfolio Dashboard is a new top-level route (`/portfolio/{entity_id}`), not a tab within the submission workbench. It operates at the commercial entity level, not the submission level.

ConcentrationPanel: Shows concentration alerts by dimension (node, pathway, derivative, cohort) with severity indicators and the list of affected portfolio entities.

SystemicNodesPanel: Visualises the highest-PageRank nodes in the Portfolio Graph. Each node shows its type, label, the number of entities that depend on it, and the estimated portfolio impact of its failure.

ScenarioPanel: Interface for running simulations. Pre-defined templates (cloud provider outage, regulatory change, supply chain disruption, vulnerability disclosure) plus custom scenario builder. Results displayed as: entity impact heatmap, aggregate loss range, mitigation recommendations.

WE-5f: Seed Script

Files to modify: `seed_dsi_bench.py`

Seed portfolio data: multiple entities assessed through the same commercial entity, with shared technology dependencies and supply chain relationships. Run concentration detection and store results. Seed at least one pre-computed scenario simulation result for the demo portfolio.

Dependencies

- Phase WE-1 complete (types, registry)
- Phase WE-3 complete (active relationships for causal edges)
- Phase WE-4 complete (CAF for portfolio-aware pricing)

- Organisational Graph Runtime (exists)

Success Criteria

1. Portfolio Graph builds correctly for seeded commercial entities
 2. Concentration alerts detect known shared dependencies in seed data
 3. Scenario simulation propagates shocks correctly through causal edges
 4. Portfolio Dashboard renders with real data from seed
 5. Appetite evaluation includes concentration warnings for relevant submissions
 6. All existing tests pass unchanged
-

Phase Dependency Chain

```
WE-1 (Foundation)
|
+---> WE-2 (Consistency Scorer)
|      |
+---> WE-3 (Discovery & Validation)
      |
      +---> WE-4 (Causal Graph Pricing)
      |
      +---> WE-5 (Portfolio Intelligence)
```

WE-2 and WE-3 can run in parallel after WE-1. WE-4 and WE-5 can run in parallel after WE-3.

Estimated Effort

Phase	Backend	Frontend	Total
WE-1: Foundation	1–2 weeks	—	1–2 weeks
WE-2: Consistency	1 week	3–4 days	~2 weeks
WE-3: Discovery	3–4 weeks	—	3–4 weeks
WE-4: CAF	2 weeks	1 week	3 weeks

WE-5: Portfolio	3–4 weeks	2 weeks	5–6 weeks
Total			14–18 weeks